Teoria Współbieżności

Kajetan Frątczak 13.11.2025 r.

Laboratorium 05

Problem pięciu filozofów

1. **Treść zadań**

   Zadanie 1

   - Problem pięciu filozofów
     - Każdy filozof zajmuje się głównie myśleniem
     - Od czasu do czasu potrzebuje zjeść
     - Do jedzenie potrzebne mu są oba widelce po jego prawej i lewej stronie
     - Jedzenie trwa skończona (ale nieokreślona z góry) ilość czasu, po czym filozof widelce odkłada i wraca do myślenia
     - Cykl powtarza się od początku

   a. Zaimplementować trywialne rozwiązanie z symetrycznymi filozofami. Zaobserwować problem blokady.
   b. Zaimplementować rozwiązanie z widelcami podnoszonymi jednocześnie. Jaki problem może tutaj wystąpić ?
   c. Zaimplementować rozwiązanie z lokajem.
   d. Wykonać pomiary dla każdego rozwiązania i wywnioskować co ma wpływ na wydajność każdego rozwiązania
   Dodatkowe zadanie: Zaproponować autorskie rozwiązanie, inne niż w/w.

2. **Argumentacja rozwiązania**

   Zadanie 1.a

   W tym rozwiązaniu każdy filozof jest symetryczny i próbuje najpierw podnieść lewy widelec, a potem prawy. Użyłem synchronizacji na poziomie widelców (metody podnies i odloz są '*synchronized*'), co zapobiega jednoczesnemu użyciu jednego widelca przez dwóch filozofów. Jednak symetria prowadzi do problemu deadlocku, czyli zakleszczenia: jeśli wszyscy filozofowie jednocześnie podniosą lewy widelec, to każdy będzie w nieskończoność czekał na prawy, który jest trzymany przez sąsiada. Wprowadziłem także losowe opóźnienie między podniesieniem lewego i prawego widelca, aby zwiększyć prawdopodobieństwo wystąpienia deadlocku.

Zadanie 1.b

Tutaj z kolei wprowadziłem jednoczesne podnoszenie widelców. Dodałem tzw. shared lock na cały stół, dzięki któremu mam pewność, że oba widelce muszą być podniesione jednocześnie wewnątrz bloku synchronized. To eliminuje problem zakleszczenia, ponieważ żaden filozof nie może podnieść tylko jednego widelca - albo bierze oba naraz, albo czeka. Jednak problemem może być teraz zagłodzenie (starvation) - jeśli jeden filozof często zdobywa lock, inni mogą czekać i czekać i czekać.... Wspólny lock serializuje dostęp do stołu, co obniża nam wydajność, bo tylko jeden filozof może jeść w danym momencie. W praktyce w wywołaniach nie zauważyłem problemu starvation, lecz to wynika ze względu na losowe czasy myślenia i jedzenia, które równoważą szanse dostępu do zasobów.

Zadanie 1.c

Rozwiązanie z lokajem (kelnerem) działa jak semafor ograniczający max liczbę filozofów przy stole do 4. Filozof musi uzyskać pozwolenie od lokaja przed podniesieniem widelców i zwolnić je po zjedzeniu. Zapobiega to deadlockowi, bo zawsze co najmniej jeden filozof może podnieść oba widelce. Eliminuje też problem zagłodzenia, bo notifyAll w *wyjdz* budzi wszystkich czekających, a losowe czasy zapewniają rotację.

Zadanie 1.d

Tu po prostu wykonujemy pomiary wydajności rozwiązań. Puściłem każde rozwiązanie na 20 sekund i zauważyłem, że w a) nastąpił deadlock po 100 razach, w b) było ok, lecz doszło jedynie do 200 razów, w c) było już lepiej i doszło do 300 razów, zaś w d) nie było sprawiedliwie, lecz najbardziej wydajnie.

Zadanie 1.dodatkowe

To rozwiązanie wprowadza hierarchię widelców na podstawie ich ponumerowania. Filozof zawsze podnosi najpierw widelec o jak najmniejszym numerze. To zapobiega deadlockowi, bo unika cyklicznego czekania - widelce są podnoszone w odgórnie ustalonej kolejności. Jednak to rozwiązanie ma problem - nie jest sprawiedliwe, gdyż filozofowie siedzący przy widelach o niższych numerach będą częściej jeść, co może prowadzić do problemu zagłodzenia innych....

3. Rozwiązanie

Zadanie 1.a

```
// 1 rozwiązanie - trywialne z symetrycznymi filozofami
// problem - deadlock (blokada)
// w tym samym momencie podniosą lewy widelec - każdy z nich czeka na prawy widelec, ale ten
jest już trzymany przez sąsiada
// nikt nie może kontynuować - wszyscy czekają wiecznie

class Widelec {
   private boolean zajety = false;
```

```java
   public synchronized void podnies() {
       // dopóki widelec jest zajęty, to reszta czeka - widelec może być zajęty na raz przez
jednego filozofa
       while (zajety){
           try {
               wait();
           } catch (InterruptedException e) {
               throw new RuntimeException(e);
           }
       }
       zajety = true;
   }
   public synchronized void odloz() {
       zajety = false;
       notify();
   }
}

// każdy filozof to osobny wątek
class Filozof extends Thread {
   private final Widelec widelecLewy, widelecPrawy;
   private int _licznik = 0;
   public Filozof(Widelec widelecLewy, Widelec widelecPrawy) {
       this.widelecLewy = widelecLewy;
       this.widelecPrawy = widelecPrawy;
   }

   public void run() {
       while (true) {
           // myślenie
           try {
               Thread.sleep((int)(Math.random()*30));
           } catch (InterruptedException e) {
               throw new RuntimeException(e);
           }
           // jedzenie
           // do jedzenia filozof podnosi oba widelce
           widelecLewy.podnies();
           // czas między podniesieniem widelca prawego i lewego
           // losowe opóźnienie - jest to po to, aby pokazać możliwość deadlocka
           try {
               if (Math.random() < 0.1) {
                   Thread.sleep(10);
               }
           } catch (InterruptedException e) {
               throw new RuntimeException(e);
           }
           widelecPrawy.podnies();

           ++_licznik;
           if (_licznik % 100 == 0) {
               System.out.println("Filozof: " + Thread.currentThread().getName() +
                       " jadlem " + _licznik + " razy");
           }
           // czas jedzenia
           try {
               Thread.sleep((int)(Math.random()*30));
           } catch (InterruptedException e) {
```

```
                throw new RuntimeException(e);
            }
            widelecLewy.odloz();
            widelecPrawy.odloz();
            // koniec jedzenia
        }
    }
}
public class Fil5mon {
    public static void main(String[] args) {
        int liczbaFilozofow = 5;
        Widelec[] widelce = new Widelec[liczbaFilozofow];
        Filozof[] filozofowie  = new Filozof[liczbaFilozofow];
        for (int i = 0; i < liczbaFilozofow; i++) {
            widelce[i] = new Widelec();
        }
        for (int i = 0; i < liczbaFilozofow; i++) {
            Widelec widelecLewy = widelce[i];
            Widelec widelecPrawy = widelce[(i+1)%liczbaFilozofow];
            filozofowie[i] = new Filozof(widelecLewy, widelecPrawy);
            filozofowie[i].start();
        }
    }
}
```

Zadanie 1.b

```
////// 2 rozwiazanie - widelce podnoszone jednocześnie
////// zapobiega to deadlockowi
////// jaki problem może tu wystąpić ? - problem zagłodzenia
class Widelec {
    private boolean zajety = false;

    public synchronized void podnies() {
        // dopóki widelec jest zajęty, to reszta czeka - widelec może być zajęty na raz przez
jednego filozofa
        while (zajety){
            try {
                wait();
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
        zajety = true;
    }
    public synchronized void odloz() {
        zajety = false;
        notify();
    }
}

// każdy filozof to osobny wątek
class Filozof extends Thread {
    private final Widelec widelecLewy, widelecPrawy;
    private int _licznik = 0;

    // shared lock na cały stół
    private final Object lock;
```

```java
    public Filozof(Widelec widelecLewy, Widelec widelecPrawy, Object lock) {
        this.widelecLewy = widelecLewy;
        this.widelecPrawy = widelecPrawy;
        this.lock = lock;
    }

    public void run() {
        while (true) {
            // myślenie
            try {
                Thread.sleep((int)(Math.random()*30));
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            // jedzenie
            // do jedzenia filozof podnosi oba widelce jednocześnie
            synchronized (lock) {
                widelecLewy.podnies();
                widelecPrawy.podnies();

                ++_licznik;
                if (_licznik % 100 == 0) {
                    System.out.println("Filozof: " + Thread.currentThread().getName() +
                            " jadlem " + _licznik + " razy");
                }
                // czas jedzenia
                try {
                    Thread.sleep((int) (Math.random() * 30));
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
                widelecLewy.odloz();
                widelecPrawy.odloz();
            }
            // koniec jedzenia
        }
    }
}

public class Fil5mon {
    public static void main(String[] args) {
        int liczbaFilozofow = 5;
        Widelec[] widelce = new Widelec[liczbaFilozofow];
        Filozof[] filozofowie  = new Filozof[liczbaFilozofow];
        Object sharedLock = new Object();

        for (int i = 0; i < liczbaFilozofow; i++) {
            widelce[i] = new Widelec();
        }

        for (int i = 0; i < liczbaFilozofow; i++) {
            Widelec widelecLewy = widelce[i];
            Widelec widelecPrawy = widelce[(i+1)%liczbaFilozofow];
            filozofowie[i] = new Filozof(widelecLewy, widelecPrawy, sharedLock);
            filozofowie[i].start();
        }
    }
}
```

Zadanie 1.c

```
// 3 rozwiazanie - z lokajem
// lokaj to taki strażnik przy stole
//// dba o to, aby max 4 filozfów na raz było przy stole
// zastosowanie lokaja powoduje usunięcie problemu zakleszczenia i zagłodzenia

class Lokaj {
   private int liczbaPrzyStole = 0;

   public synchronized void wejdz() throws InterruptedException {
       while (liczbaPrzyStole >= 4) {
           wait();
       }
       liczbaPrzyStole++;
   }
   public synchronized void wyjdz() throws InterruptedException {
       liczbaPrzyStole--;
       notifyAll();
   }
}

class Widelec {
   private boolean zajety = false;

   public synchronized void podnies() {
       // dopóki widelec jest zajęty, to reszta czeka - widelec może być zajęty na raz przez
jednego filozofa
       while (zajety){
           try {
               wait();
           } catch (InterruptedException e) {
               throw new RuntimeException(e);
           }
       }
       zajety = true;
   }
   public synchronized void odloz() {
       zajety = false;
       notify();
   }
}

// każdy filozof to osobny wątek
class Filozof extends Thread {
   private final Widelec widelecLewy, widelecPrawy;
   private int _licznik = 0;
   private final Lokaj lokaj;

   public Filozof(Widelec widelecLewy, Widelec widelecPrawy, Lokaj lokaj) {
       this.widelecLewy = widelecLewy;
       this.widelecPrawy = widelecPrawy;
       this.lokaj = lokaj;
   }

   public void run() {
       while (true) {
           // myślenie
           try {
```

```java
                Thread.sleep((int)(Math.random()*30));
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

            // prośba o pozwolenia na jedzenie
            try {
                lokaj.wejdz();
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

            // jedzenie
            widelecLewy.podnies();
            widelecPrawy.podnies();
            ++_licznik;
            if (_licznik % 100 == 0) {
                System.out.println("Filozof: " + Thread.currentThread().getName() +
                        " jadlem " + _licznik + " razy");
            }
            // czas jedzenia
            try {
                Thread.sleep((int)(Math.random()*30));
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

            widelecLewy.odloz();
            widelecPrawy.odloz();
            // zwolnienie miejsca przy stole
            try {
                lokaj.wyjdz();
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            // koniec jedzenia
        }
    }
}

public class Fil5mon {
    public static void main(String[] args) {
        int liczbaFilozofow = 5;
        Widelec[] widelce = new Widelec[liczbaFilozofow];
        Filozof[] filozofowie  = new Filozof[liczbaFilozofow];
        Lokaj lokaj = new Lokaj();

        for (int i = 0; i < liczbaFilozofow; i++) {
            widelce[i] = new Widelec();
        }

        for (int i = 0; i < liczbaFilozofow; i++) {
            Widelec widelecLewy = widelce[i];
            Widelec widelecPrawy = widelce[(i+1)%liczbaFilozofow];
            filozofowie[i] = new Filozof(widelecLewy, widelecPrawy,  lokaj);
            filozofowie[i].start();
        }
    }
}
```

Zadanie 1.dodatkowe

```
// 4 rozwiązanie - autorskie
// ustalam odgórnie pewną wartość widelca (od 1 do 5) (taka hierarchia)
// filozof zawsze będzie chciał podnieść widelec o jak najmniejszym numerze
// deadlocka nie będzie dzięki takiemu rozwiązaniu
// tylko chyba może w tym przypadku dojść do zagłodzenia, a na pewno nie jest to sprawiedliwe
rozwiązanie, ponieważ
// jedni będą jeść częściej niż inni

class Widelec {
   private boolean zajety = false;
   private final int numer;

   public Widelec(int numer) {
       this.numer = numer;
   }

   public int getNumer() {
       return numer;
   }

   public synchronized void podnies() {
       // dopóki widelec jest zajęty, to reszta czeka - widelec może być zajęty na raz przez
jednego filozofa
       while (zajety){
           try {
               wait();
           } catch (InterruptedException e) {
               throw new RuntimeException(e);
           }
       }
       zajety = true;
   }
   public synchronized void odloz() {
       zajety = false;
       notify();
   }
}

// każdy filozof to osobny wątek
class Filozof extends Thread {
   private final Widelec widelecLewy, widelecPrawy;
   private int _licznik = 0;

   public Filozof(Widelec widelecLewy, Widelec widelecPrawy) {
       this.widelecLewy = widelecLewy;
       this.widelecPrawy = widelecPrawy;
   }

   public void run() {
       while (true) {
           // myślenie
           try {
               Thread.sleep((int)(Math.random()*30));
           } catch (InterruptedException e) {
               throw new RuntimeException(e);
           }
```

```java
            // wybieramy który widelec podnosimy jako pierwszy
            Widelec pierwszy, drugi;
            if (widelecLewy.getNumer() < widelecPrawy.getNumer()) {
                pierwszy = widelecLewy;
                drugi = widelecPrawy;
            } else{
                pierwszy = widelecPrawy;
                drugi = widelecLewy;
            }

            pierwszy.podnies();
            // czas między podniesieniem widelca prawego i lewego
            try {
                if (Math.random() < 0.1) {
                    Thread.sleep(10);
                }
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            drugi.podnies();

            ++_licznik;
            if (_licznik % 100 == 0) {
                System.out.println("Filozof: " + Thread.currentThread().getName() +
                        " jadlem " + _licznik + " razy");
            }
            // czas jedzenia
            try {
                Thread.sleep((int)(Math.random()*30));
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

            // odkładnie widelców - kolejność bez znaczenia
            widelecLewy.odloz();
            widelecPrawy.odloz();
            // koniec jedzenia
        }
    }
}

public class Fil5mon {
    public static void main(String[] args) {
        int liczbaFilozofow = 5;
        Widelec[] widelce = new Widelec[liczbaFilozofow];
        Filozof[] filozofowie  = new Filozof[liczbaFilozofow];

        for (int i = 0; i < liczbaFilozofow; i++) {
            widelce[i] = new Widelec(i+1);
        }

        for (int i = 0; i < liczbaFilozofow; i++) {
            Widelec widelecLewy = widelce[i];
            Widelec widelecPrawy = widelce[(i+1)%liczbaFilozofow];
            filozofowie[i] = new Filozof(widelecLewy, widelecPrawy);
            filozofowie[i].start();
        }
    }
}
```

4. Wykresy, tabele, wyniki

Zadanie 1.a
Przykład deadlocka

Filozof 0	Filozof 1	Filozof 2	Filozof 3	Filozof 4
bierze lewy (0)	bierze lewy (1)	bierze lewy (2)	bierze lewy (3)	bierze lewy (4)
czeka na prawy (1)	czeka na prawy (2)	czeka na prawy (3)	czeka na prawy (4)	czeka na prawy (0)

Rysunek 1: Wywołanie programu 1.a

Zadanie 1.b
Przykład zagłodzenia

F0	F1	F2	F3	F4
myślenie	myślenie	myślenie	myślenie	myślenie
wchodzi do `synchronized`	myślenie	myślenie	myślenie	myślenie
podnosi widelce	myślenie	myślenie	myślenie	myślenie
je	wchodzi do `synchronized`	myślenie	myślenie	myślenie
odkłada	podnosi widelce	myślenie	myślenie	myślenie
myślenie	je	wchodzi do `synchronized`	myślenie	myślenie
...	...	...	...	F4 ciągle czeka

Fragment wywołania

Filozof: Thread-2 jadlem 100 razy
Filozof: Thread-4 jadlem 100 razy
Filozof: Thread-3 jadlem 100 razy
Filozof: Thread-0 jadlem 100 razy
Filozof: Thread-1 jadlem 100 razy
Filozof: Thread-2 jadlem 200 razy
Filozof: Thread-3 jadlem 200 razy
Filozof: Thread-0 jadlem 200 razy
Filozof: Thread-4 jadlem 200 razy
Filozof: Thread-1 jadlem 200 razy
Filozof: Thread-2 jadlem 300 razy
Filozof: Thread-3 jadlem 300 razy
Filozof: Thread-0 jadlem 300 razy
Filozof: Thread-4 jadlem 300 razy
Filozof: Thread-1 jadlem 300 razy

Nie dochodzi w moim przypadku do zagłodzenia m. in. przez to, że mamy losowe czasy jedzenia i myślenia, co automatycznie nam ogranicza możliwość zagłodzenia filozofów.

Zadanie 1.c

Fragment wywołania (już bez zakleszczenia oraz zagłodzenia)

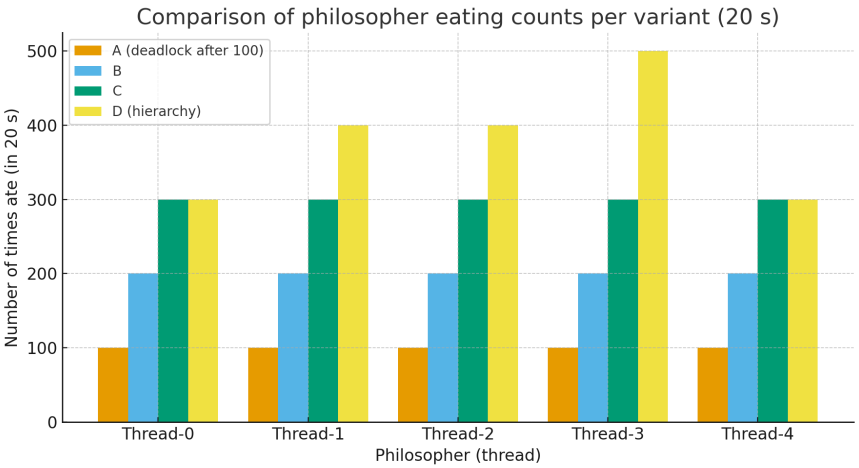
Filozof: Thread-2 jadlem 100 razy
Filozof: Thread-4 jadlem 100 razy
Filozof: Thread-1 jadlem 100 razy
Filozof: Thread-3 jadlem 100 razy
Filozof: Thread-0 jadlem 100 razy
Filozof: Thread-2 jadlem 200 razy
Filozof: Thread-1 jadlem 200 razy
Filozof: Thread-4 jadlem 200 razy
Filozof: Thread-3 jadlem 200 razy
Filozof: Thread-0 jadlem 200 razy
Filozof: Thread-2 jadlem 300 razy
Filozof: Thread-1 jadlem 300 razy
Filozof: Thread-4 jadlem 300 razy
Filozof: Thread-3 jadlem 300 razy
Filozof: Thread-0 jadlem 300 razy
Filozof: Thread-2 jadlem 400 razy
Filozof: Thread-4 jadlem 400 razy
Filozof: Thread-1 jadlem 400 razy
Filozof: Thread-0 jadlem 400 razy
Filozof: Thread-3 jadlem 400 raz

Zadanie 1.dodatkowe

Fragment wywołania

Filozof: Thread-3 jadlem 100 razy
Filozof: Thread-2 jadlem 100 razy
Filozof: Thread-1 jadlem 100 razy
Filozof: Thread-4 jadlem 100 razy
Filozof: Thread-0 jadlem 100 razy
Filozof: Thread-3 jadlem 200 razy
Filozof: Thread-2 jadlem 200 razy
Filozof: Thread-1 jadlem 200 razy
Filozof: Thread-4 jadlem 200 razy
Filozof: Thread-0 jadlem 200 razy
Filozof: Thread-3 jadlem 300 razy
Filozof: Thread-2 jadlem 300 razy
Filozof: Thread-1 jadlem 300 razy
Filozof: Thread-4 jadlem 300 razy
Filozof: Thread-0 jadlem 300 razy
Filozof: Thread-3 jadlem 400 razy
Filozof: Thread-2 jadlem 400 razy
Filozof: Thread-1 jadlem 400 razy
Filozof: Thread-3 jadlem 500 razy
Filozof: Thread-4 jadlem 400 razy
Filozof: Thread-0 jadlem 400 razy
Filozof: Thread-2 jadlem 500 razy
Filozof: Thread-1 jadlem 500 razy
Filozof: Thread-3 jadlem 600 razy
Filozof: Thread-4 jadlem 500 razy
Filozof: Thread-2 jadlem 600 razy
Filozof: Thread-0 jadlem 500 razy
Filozof: Thread-1 jadlem 600 razy
Filozof: Thread-3 jadlem 700 razy
Filozof: Thread-4 jadlem 600 razy
Filozof: Thread-2 jadlem 700 razy
Filozof: Thread-0 jadlem 600 razy
Filozof: Thread-3 jadlem 800 razy
Filozof: Thread-1 jadlem 700 razy
Filozof: Thread-2 jadlem 800 razy
Filozof: Thread-4 jadlem 700 razy
Filozof: Thread-3 jadlem 900 razy
Filozof: Thread-1 jadlem 800 razy
Filozof: Thread-0 jadlem 700 razy
Filozof: Thread-2 jadlem 900 razy
Filozof: Thread-3 jadlem 1000 razy
Filozof: Thread-4 jadlem 800 razy
Filozof: Thread-1 jadlem 900 razy
Filozof: Thread-0 jadlem 800 razy
Filozof: Thread-3 jadlem 1100 razy
Filozof: Thread-2 jadlem 1000 razy
Filozof: Thread-4 jadlem 900 razy
Filozof: Thread-1 jadlem 1000 razy
Filozof: Thread-0 jadlem 900 razy

Wykres (porównanie po 20 sek. od uruchomienia programu) - zadanie 1.d



Rysunek 2: Wykres wydajności dla 20 sekund po uruchomieniu

5. Wnioski

- Problem pięciu filozofów ilustruje klasyczne wyzwania współbieżności: deadlock oraz starvation. Symetryczne rozwiązanie jest podatne na oba problemy, wprowadzenie globalnego locka i dzięki temu synchronizacji podnoszenia widelców eliminuje deadlock, lecz nie starvation, zaś wprowadzenie lokaja skutecznie zapobiega wymienionym wcześniej problemom. Za to dodatkowe rozwiązanie jest szybkie i proste, lecz nieskuteczne wobec problemu starvation oraz nie jest sprawiedliwe.
- Wydajność rozwiązania zależy głównie od współbieżności, czyli tego ilu filozofów może na raz jeść oraz od wystąpienia deadlocka bądź zagłodzenia. W rozwiązaniu a) zasadniczo nie ma co testować, ponieważ po deadlocku jest to wydajność zerowa. W b) ograniczamy się do max 1 jedzącego na raz, co ogranicza wydajność, w c) możemy mieć 4 jedzących co poprawia wydajność oraz rozwiązuje potencjalne problemy, zaś w d) mamy wysoką współbieżność, lecz niską sprawiedliwość rozwiązania.
- Ogólnie, przez losowe czasy jedzenia i myślenia oraz potencjalnie problemy, jak deadlock, ciężko sensownie przetestować te rozwiązania.

6. Bibliografia

https://pl.wikipedia.org/wiki/Problem_ucztuj%C4%85cych_filozof%C3%B3w
https://diningphilosophers.eu/hierarchy_asymmetric/